SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 - Architecture Design Assignment

Course Code	CSA10 / CSA1063	**Course Title**	Software Engineering
CO Assessed	CO2	**Assessment**	Assessment Tool 2 - Architecture Design Assignment
Weightage	20%	**Total Marks**	25
Student Name	V SHASHANK SAI	**Reg. No.**	192524509
Date	27/10/26	**Duration**	60 minutes

CO2 Statement: Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills.

Code of Conduct

I V.Shashank sai (192524509) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: V.shashank sai

Annexure A - Architecture Design Assignment

Architecture Design of "MediQueue" - A Cloud-Based Smart Hospital Appointment and Patient Queue Management System

Problem Statement: Multi-speciality hospitals still depend on paper tokens, phone-in bookings and crowded corridors, so patients wait for hours without knowing their turn, doctors lose consulting time to no-shows and walk-in chaos, and management has no live view of outpatient (OPD) load. The hospital needs a secure, scalable and highly available platform that lets patients book appointments online, check in by QR code, track their position in a live queue, get SMS / WhatsApp alerts, and lets reception, doctors and administrators manage priority cases, delays and cancellations in real time - while integrating with the hospital information system (HIS / EMR), payment gateway and national health ID (ABHA), and protecting patient data.

1. Analyze System Requirements

1.1 Study the problem statement and identify the system objectives

MediQueue is a **real-time, multi-role, privacy-critical** platform. Demand is concentrated: most appointments are booked in the evening and early morning, and the OPD floor is busiest between 8 AM and 12 noon when hundreds of patients check in at the same time and every status change (token called, doctor delayed, patient skipped) must reach many screens within seconds. A double-booked slot, a wrong token order or a leaked medical record directly harms patients and the hospital's reputation.

1. **Easy appointment booking:** search by department / doctor / date, book, reschedule and cancel without double-booking; slots are locked briefly while a patient completes booking.
2. **Fair, transparent queueing:** issue a digital token at check-in, order patients by appointment time and clinical priority (emergency, senior citizen, pregnant, differently-abled), and show live position and estimated waiting time.
3. **Real-time communication:** push token calls, delays and doctor-availability changes to the patient app, waiting-room display boards and staff consoles within 2 seconds.
4. **Reduce no-shows and idle time:** automated reminders, grace-period handling, auto-reassignment of unused slots and waitlist promotion.
5. **Operational control:** let reception and doctors handle walk-ins, priority override, skip / recall, doctor delay and room change from role-based consoles.

6. **Integration:** exchange patient, doctor and encounter data with the HIS / EMR, collect payments online, and link ABHA health IDs.
7. **Insight and compliance:** wait-time analytics, doctor utilisation, no-show rates, and a tamper-evident audit trail that respects the Digital Personal Data Protection Act, 2023 and ABDM guidelines.
8. **Extensibility:** add new departments, hospitals (branches), teleconsultation or lab-queue modules without rewriting the platform.

1.2 Functional requirements that influence the architecture

ID	Functional Requirement	Architectural Influence
FR1	Register patients (mobile OTP / ABHA ID), staff login, roles: patient, receptionist, doctor, nurse, admin, auditor	Separate Identity & Access service with OTP + OIDC, RBAC and short-lived JWT checked at the gateway
FR2	Maintain departments, doctors, shifts, consultation rooms and generate bookable slots	Doctor & Schedule service owning its own database; slot generation as a scheduled job
FR3	Book / reschedule / cancel appointments with no double-booking; waitlist	Appointment service with optimistic locking and short Redis slot-locks; events for waitlist promotion
FR4	Check-in by QR, kiosk or reception; issue token; apply priority rules; call next, skip, recall, no-show	Dedicated Queue Engine (command side) with an ordered, in-memory queue per doctor; deterministic priority rules
FR5	Show live position, estimated wait and 'now serving' on app, kiosk and TV boards	CQRS read model (Queue View) + Realtime Gateway using WebSocket / SSE push instead of polling
FR6	Send booking confirmation, reminders, 'your turn is near' and delay alerts by SMS / WhatsApp / push	Notification service consuming domain events; external gateways; retry and de-duplication
FR7	Billing, online payment, receipts; sync patient and encounter data with HIS / EMR	Billing service with payment webhooks; HIS Integration Adapter (anti-corruption layer, FHIR / HL7)
FR8	Dashboards (OPD load, waiting time, no-show rate), reports and immutable audit trail	Read-optimised analytics store fed by events; append-only Audit service

1.3 Non-functional requirements that influence the architecture

Category	Measurable Requirement	Architectural Influence
Performance	Booking API p95 < 400 ms; check-in and token issue < 1 s; token-call push to all screens < 2 s; app home screen load < 3 s on 4G	Redis for live queues and slot locks, CDN for static assets, WebSocket push, connection pooling
Scalability	2,000 concurrent users per hospital at OPD peak; 20,000 across a 10-branch network; scale out only in 8 AM - 12 noon and evening booking peaks	Stateless services, horizontal auto-scaling, partitioned event topics per branch
Availability	99.9% overall; 99.95% for check-in and queue services during OPD hours (7 AM - 9 PM); no single point of failure	Multi-AZ deployment, health checks, replicated DB / Redis / broker, graceful degradation
Reliability / Durability	No lost booking or token; RPO <= 5 min, RTO <= 30 min; queue state rebuilt from events after a crash	Write-ahead log, event replay, replicated storage, idempotent APIs, nightly backups
Security & Privacy	Encrypted personal health data (TLS 1.3, AES-256); OTP + MFA for staff; RBAC; consent records; protection against OWASP Top 10 and OTP abuse	WAF, API gateway, secrets vault, field-level encryption, least-privilege DB roles
Interoperability	Standards-based integration with HIS / EMR (HL7 FHIR), ABDM, payment gateway and SMS providers	Adapter / anti-corruption layer, versioned REST APIs, webhooks
Usability / Accessibility	Works for first-time and elderly users; English, Tamil and Hindi; large-font kiosk mode; low-bandwidth friendly	Lightweight PWA, i18n, simple flows, SMS fallback for users without smartphones
Maintainability	New department rule or notification channel released independently; >= 80% unit-test coverage	Domain-based service boundaries, CI/CD, contract tests, feature flags

Category	Measurable Requirement	Architectural Influence
Auditability / Compliance	Every login, record access, token override and billing change logged immutably; retention per hospital policy and DPDP Act	Append-only hash-chained audit store, retention policies, restricted access

1.4 Key quality attributes (ranked by influence on the architecture)

Rank	Quality Attribute	Why it dominates in MediQueue
1	Availability & Reliability	A hospital cannot pause. If check-in or the queue is down, corridors overflow and urgent patients may be delayed; token order must never be lost.
2	Security & Privacy	The system stores identity, contact and visit information of patients; leaks cause legal, ethical and reputational damage.
3	Performance (real-time)	The value of MediQueue is *live* information; a queue board that lags by minutes is worse than none.
4	Interoperability	The platform is useless if it cannot share data with the existing HIS / EMR, payment and ABDM systems.
5	Usability	Users include elderly, first-time and low-literacy patients; a confusing flow defeats the purpose.
6	Scalability & Maintainability	Predictable daily peaks and growth to more branches and departments require elastic, independently evolvable parts.

2. Select a Suitable Software Architecture

2.1 Selected architectural style

Event-Driven Microservices with CQRS (Command Query Responsibility Segregation) and Real-Time Push, fronted by an API Gateway with Backend-for-Frontend (BFF) layers.

Patients and staff use channel-specific clients (mobile PWA, kiosk, consoles, TV boards). A secured edge (CDN, WAF, API gateway, BFFs and a Realtime Gateway) routes requests to a small set of coarse-grained, domain-aligned services - Appointment, Doctor & Schedule, Patient Profile, Check-in, Queue Engine, Notification, Billing, Analytics and Audit. The **queue domain is split by CQRS**: the Queue Engine handles commands (issue token, call next, skip, priority override) and writes events, while a separate Queue View keeps a fast read model that is pushed live to screens. Services communicate synchronously for request-response needs and **asynchronously through an event bus (Apache Kafka)** for everything else. Each service owns its database, and the platform runs on containers (Kubernetes) across multiple availability zones.

2.2 Evaluation of candidate architectural styles

Style	Burst scalability	Fault isolation	Independent evolution	Verdict
Single monolith (3-tier MVC)	Poor - whole app scales together	Poor - one fault stops booking and queue	Low	Rejected: a reporting spike or bug can freeze the OPD queue
Modular monolith	Medium	Medium - single deployable	Medium	Rejected for core, but its clear modules guide the service boundaries
Serverless (FaaS) only	Good	Good	Good	Rejected: cold starts and no long-lived WebSocket connections for live boards; used only for scheduled jobs
SOA with central ESB	Medium	Medium - ESB is a bottleneck	Medium	Rejected: heavy middleware, single choke point
Microservices with synchronous REST only	Good	Poor - cascading failures	Good	Rejected: Notification or Billing slowness would block check-in

Style	Burst scalability	Fault isolation	Independent evolution	Verdict
Event-Driven Microservices + CQRS + Push	Excellent - scale queue and gateway independently	Excellent - isolated failures, replayable events	Excellent	**Selected**

2.3 Why the selected architecture is the most suitable

- **Reads far outnumber writes in the queue.** Hundreds of waiting patients refresh their position for every single 'call next'. CQRS lets the read model be scaled and cached separately, while the command side stays small, ordered and consistent.
- **Real-time is a first-class need.** Event-driven design plus WebSocket / SSE push delivers token calls to every screen without polling, which also protects servers at peak time.
- **Fault isolation protects care delivery.** If Notification, Billing or Analytics fails, check-in and queueing continue; events wait in Kafka and are processed after recovery.
- **Right data store for each job:** relational data (appointments, schedules, bills) in PostgreSQL, volatile queue state and slot locks in Redis, documents in object storage, and an immutable audit store.
- **Independent evolution:** a new priority rule, WhatsApp channel or teleconsultation module is added as a new service or event consumer without touching booking.
- **Right-sized complexity:** only about 9 coarse-grained services, managed Kubernetes / Kafka and shared templates keep operations manageable for a hospital IT team.

3. Draw the Software Architecture Diagram

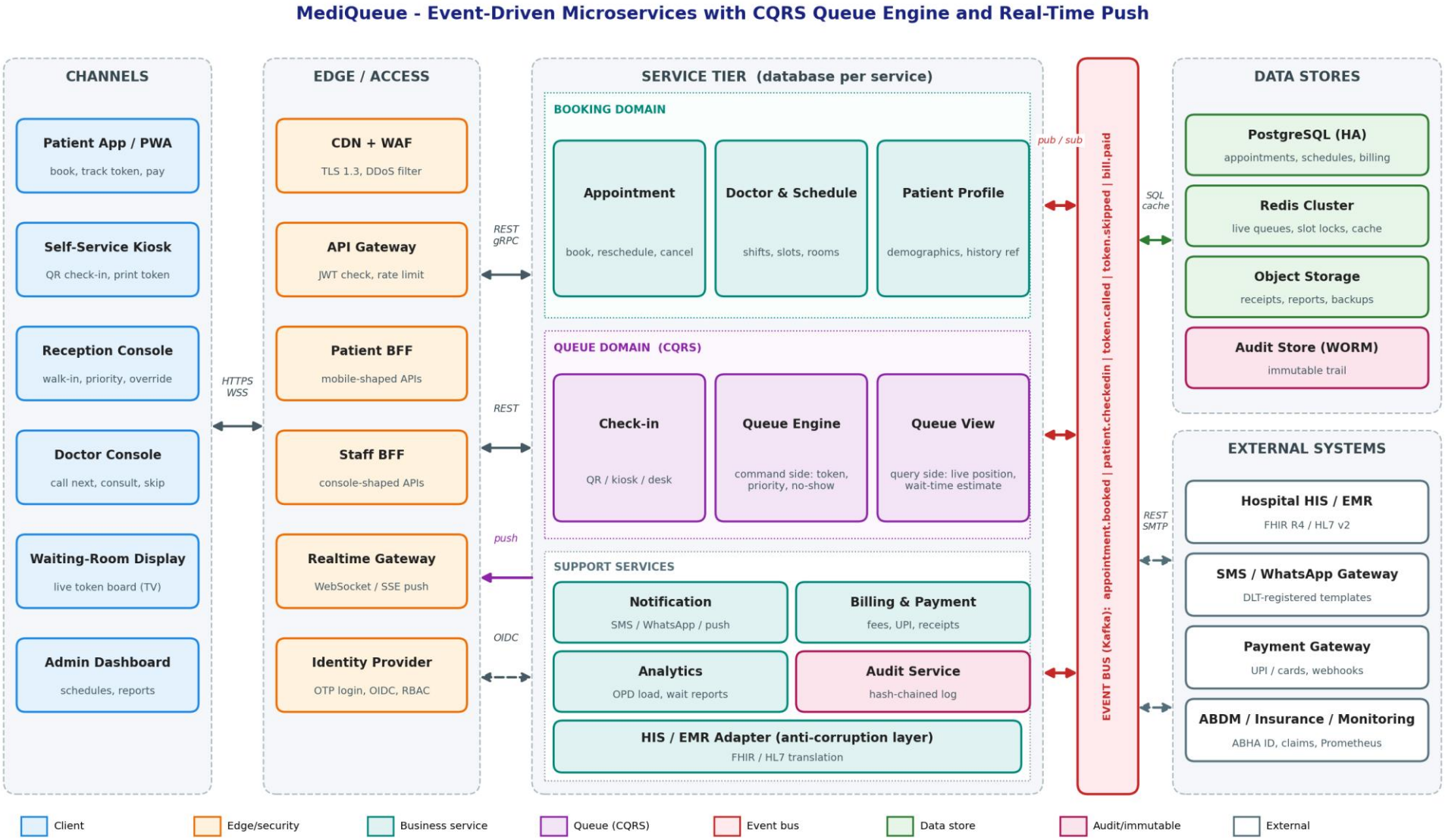


Figure 1: Event-driven microservice architecture of the MediQueue platform (CQRS queue domain)

3.1 Notation and reading guide

- **Columns (left to right):** Channels, Edge / Access, Service Tier, Event Bus, and Data Stores / External Systems. A request travels left to right; asynchronous events flow through the vertical event bus.
- **Colours:** blue = client channels, orange = edge and security components, teal = business services, purple = queue (CQRS) services, red = event bus, green = data stores, pink = audit / immutable components, white = external systems.
- **Arrows:** solid double-headed = synchronous request-response (HTTPS / REST / gRPC / WSS); red arrows to the bus = publish / subscribe domain events; purple single arrow = real-time push from Queue View to Realtime Gateway; dashed = external or security integrations (OIDC, SMTP, REST, webhooks).
- **Dotted boxes** inside the Service Tier group services by domain: Booking, Queue (CQRS) and Support.

4. Explain Components and Their Interactions

4.1 Purpose and responsibilities of each component

Component	Responsibility	Technology (proposed)
Patient App / PWA	Search doctors, book / cancel, QR check-in, live token and wait-time view, payment, reminders	React PWA, Service Worker, i18n
Self-Service Kiosk	QR / mobile-number check-in and token printing at hospital entrance; large-font, multilingual UI	Browser kiosk app, thermal printer API
Reception & Doctor Consoles	Walk-in registration, priority override, call next / skip / recall, mark doctor delay	React SPA, WebSocket client
Waiting-Room Display	Shows 'now serving' and next tokens per doctor on TV screens	Lightweight web client over SSE
Admin Dashboard	Doctor schedules, department settings, analytics, user management	React SPA, charts
CDN + WAF	Serves static assets, filters malicious traffic, absorbs DDoS	CloudFront / NGINX, AWS WAF
API Gateway	Terminates TLS, validates JWT, rate-limits (esp. OTP and booking), routes to BFFs / services	Kong / Spring Cloud Gateway
Patient BFF / Staff BFF	Aggregate several service calls into one channel-friendly response; keep mobile payloads small	Node.js (Express / NestJS)
Realtime Gateway	Holds WebSocket / SSE connections and fans out token and delay updates to subscribed screens	Socket.IO / SSE behind L7 load balancer, Redis pub/sub
Identity & Access (+ IdP)	Mobile OTP login, ABHA linking, staff OIDC + MFA, RBAC, consent records, short-lived tokens	Keycloak, JWT
Appointment Service	Book, reschedule, cancel; prevents double-booking with slot locks; manages waitlist	Spring Boot, PostgreSQL, Redis
Doctor & Schedule Service	Departments, doctors, shifts, leave, rooms; generates slots	Spring Boot, PostgreSQL
Patient Profile Service	Demographics, contact, consent and ABHA reference (clinical data stays in HIS / EMR)	Spring Boot, PostgreSQL (encrypted fields)
Check-in Service	Validates appointment / QR, marks arrival, requests a token	Node.js, PostgreSQL
Queue Engine (command side)	Issues tokens, applies priority and grace-period rules, handles call / skip / no-show; emits queue events	Java / Go, Redis sorted sets, Kafka
Queue View (query side)	Maintains read model of position and estimated wait per patient; pushes changes to Realtime Gateway	Redis, event consumer
Notification Service	Sends SMS / WhatsApp / push for confirmations, reminders and 'your turn is near' with retry and de-duplication	Node.js workers, SMS / WhatsApp APIs
Billing & Payment Service	Consultation fee, online payment, receipts, refund on cancellation	Spring Boot, payment gateway SDK

Component	Responsibility	Technology (proposed)
Analytics Service	Builds OPD load, waiting-time and no-show reports from events	Kafka consumers, warehouse / read store
Audit Service & Store	Writes every critical action to an append-only, hash-chained store	WORM object store / append-only table
HIS / EMR Integration Adapter	Translates between MediQueue and hospital HIS / EMR models; isolates changes	FHIR / HL7 adapters, REST
Event Bus	Durable, replayable events: appointment.booked, patient.checkedin, token.called, token.skipped, bill.paid	Apache Kafka
Data stores	PostgreSQL (HA), Redis cluster, object storage, audit store	PostgreSQL, Redis, S3 / MinIO

4.2 How components communicate and exchange data

From - To	Mechanism	Data exchanged
Client - Gateway	HTTPS / REST (JSON), TLS 1.3	Login, search, booking, check-in, payment calls carrying JWT
Client - Realtime Gateway	WSS / SSE	Token calls, position changes, delay and cancellation alerts
Gateway - BFF - Services	REST / gRPC with service discovery	Authenticated requests with user claims and tenant (branch) id
Services - Identity Provider	OIDC / JWT validation	Tokens, roles, public keys (JWKS)
Services - Event Bus	Kafka publish / subscribe	Domain events (appointment.booked, patient.checkedin, token.called, token.skipped, bill.paid, notify.request)
Queue Engine - Queue View	Events via Kafka	Queue changes applied to read model; View pushes update to Realtime Gateway
Services - Data stores	SQL, Redis protocol, S3 API	Appointments, schedules, live queue state, receipts
Integration Adapter - HIS / EMR	REST (FHIR / HL7) over HTTPS	Patient and doctor master data in; encounter status out
Billing - Payment Gateway	REST + signed webhooks	Payment requests, success / failure callbacks
Notification - SMS / WhatsApp	REST / SMTP	Confirmations, reminders, alerts
All services - Audit Service	Async events	Who did what, when, from where; hash of previous record

4.3 Overall workflow of the system

Schedule setup -> Patient login (OTP) -> Book slot -> Reminder -> Arrival and QR check-in -> Token and priority -> Live queue + call -> Consultation -> Billing -> Feedback and analytics

1. **Setup:** The admin configures departments, doctors, shifts and consultation rooms; the adapter syncs doctor and patient master data from the HIS; the Schedule service publishes bookable slots.
2. **Login:** The patient opens the PWA from the CDN and logs in with a mobile OTP (or ABHA ID); the Identity service issues a short-lived JWT validated at the gateway.
3. **Book:** The patient selects a doctor and slot; Appointment service takes a 2-minute Redis lock, confirms the booking in PostgreSQL and publishes appointment.booked; payment is collected if required.
4. **Remind:** Notification service sends confirmation immediately and reminders 24 hours and 2 hours before; unconfirmed slots can be released to the waitlist.
5. **Check-in:** On arrival the patient scans the QR at the kiosk (or reception registers a walk-in); Check-in service validates and requests a token; patient.checkedin is published.
6. **Token and priority:** Queue Engine adds the patient to the doctor's ordered queue using appointment time, grace period and priority category; emergency cases are escalated by reception.

7. **Live queue:** Queue View recalculates position and estimated wait and the Realtime Gateway pushes updates to the app, TV board and consoles; Notification sends 'your turn is near' for the patient three places away.
8. **Consultation:** The doctor presses 'call next'; token.called is published; if the patient does not appear within the grace period the token is skipped and can be recalled.
9. **Billing and sync:** Billing generates the receipt; the adapter sends encounter status to the HIS / EMR.
10. **Analytics and audit:** Events update the analytics store (wait times, no-shows) and every step writes an immutable audit record.

5. Discuss Quality Attributes

Attribute	How the architecture addresses it (tactics)	Target / evidence
Scalability	Stateless services behind a load balancer scale horizontally (Kubernetes HPA on CPU and request rate); Kafka partitions per branch let consumers scale with lag; read-heavy queue view scales independently of the command side; Redis absorbs hot reads; CDN offloads static content; scheduled pre-scaling before 8 AM and evening booking peaks.	2,000 concurrent users per hospital now; 20,000+ across branches by adding replicas and partitions
Security & Privacy	OTP + OIDC login, MFA for staff; JWT with short expiry; RBAC and branch-level tenancy; TLS 1.3 in transit, AES-256 at rest with field-level encryption of identifiers; WAF, rate limiting on OTP / booking; secrets vault; consent capture and data minimisation (clinical data stays in HIS); hash-chained audit log; per-service least-privilege DB accounts.	OWASP Top 10 controls; DPDP Act 2023 and ABDM aligned; tamper-evident logs
Performance	Redis for live queues and slot locks; read-model (CQRS) avoids heavy joins; WebSocket push rather than polling; CDN for assets; async persistence of derived data; connection pooling and indexed queries; BFFs reduce mobile payload size.	Booking p95 < 400 ms; token push < 2 s; check-in < 1 s
Reliability	Idempotent APIs with request ids; optimistic locking against double-booking; queue state rebuilt from Kafka events; retries with back-off and circuit breakers; DB replication, WAL archiving and nightly backups; graceful degradation (manual token mode at reception if the app is down; SMS fallback if WhatsApp fails).	No lost booking or token; RPO <= 5 min; RTO <= 30 min
Maintainability	Services aligned to business domains with small codebases; database-per-service; versioned REST / event contracts; independent CI/CD with canary releases; externalised configuration and feature flags; centralised logs, metrics, tracing; contract and integration tests; infrastructure as code.	A change to Notification can be released without touching Queue; fast fault diagnosis
Availability	Multi-AZ deployment with at least two replicas per service; health checks and automatic restart; load balancer failover; managed PostgreSQL with automatic failover; replicated Kafka and Redis; DR region; blue-green deployment outside OPD hours; 24x7 monitoring and alerting.	99.9% overall; 99.95% for check-in and queue in OPD hours

6. Justify Architectural Decisions

6.1 Rationale behind the architectural choices

ID	Decision	Rationale	Alternative considered
D1	Domain-based microservices (about 9) instead of a monolith	Queue and gateway scale and fail independently of billing and reporting; branches can be added without redesign	Modular monolith (simpler, but one deployable and shared failure)
D2	CQRS for the queue domain	Read traffic is much higher than write traffic; separate read model gives fast, cacheable live views while commands stay ordered	Single queue table queried by everyone (contention, slow screens)
D3	Event bus (Kafka) for notifications, analytics, audit and billing events	Decouples slow tasks from the check-in path; durable, replayable, smooths bursts	Direct synchronous calls (cascading failures); simple queue (no replay)
D4	WebSocket / SSE push via a Realtime Gateway	Instant updates with fewer requests than polling; better battery and server load	Client polling every few seconds (lag and load)

ID	Decision	Rationale	Alternative considered
D5	API Gateway + BFFs + external IdP (OIDC)	Central security and rate limiting; channel-specific responses; no password storage for patients	Authentication coded in each service; one generic API for all channels
D6	Database-per-service with polyglot storage	Each data type gets suitable consistency, speed and cost; loose coupling	One shared database (tight coupling, contention)
D7	HIS / EMR Integration Adapter (anti-corruption layer)	Isolates MediQueue from HIS model changes and vendor differences; one place to change	Direct DB access to HIS (fragile and insecure)
D8	Separate immutable Audit Service	Proof for disputes, medico-legal queries and accreditation; separation of duty	Application logs in the main DB (editable)
D9	Containers on Kubernetes across multiple AZs	Auto-scaling, self-healing, rolling updates, portable across cloud / on-premise	Fixed VMs (over-provisioned, manual scaling)

6.2 Trade-offs considered

Trade-off	Cost accepted	Mitigation
Microservices vs. simplicity	Higher operational complexity, network latency, distributed debugging	Limit to about 9 coarse-grained services; managed Kubernetes / Kafka; distributed tracing; shared templates
Eventual vs. strong consistency	Position / wait-time estimates and reports may lag by a second or two	Strong consistency for booking and token issue; eventual for derived views, shown with 'updated just now' status
Privacy vs. convenience	SMS and app alerts expose some visit details; mobile OTP is weaker than staff MFA	Minimal message content (token number only), consent and opt-out, masked identifiers, rate-limited OTP
Cost vs. availability	Multi-AZ replicas and DR region increase cost	Scale down outside OPD hours; reserve capacity only for known peaks
Automation vs. clinical judgement	Auto-ordering may not suit urgent cases	Reception / doctor priority override with mandatory reason, fully audited
Realtime push vs. connection load	Many long-lived connections consume resources	Dedicated Realtime Gateway scaled on connection count; SSE fallback and SMS for non-app users

6.3 Support for future enhancements, integration and long-term maintainability

- **New features as new services or consumers:** teleconsultation, lab and pharmacy queues, AI-based wait-time prediction, and digital prescriptions can subscribe to existing Kafka topics without modifying current services (open-closed principle).
- **Replaceable intelligence:** the wait-time estimator is an event consumer behind a stable contract, so simple averages can later be replaced by a machine-learning model and A/B tested.
- **Integration-ready:** versioned REST APIs, FHIR / HL7 and OIDC allow other hospital systems, insurance portals, lab systems and government health platforms to be added through adapters.
- **Multi-branch growth:** branch identifiers in tokens and data partitions let several hospitals share one deployment with isolated data.
- **Evolution path:** services are independently versioned; a rarely used service can be merged or a hot one split further; containers avoid cloud-vendor lock-in.
- **Operational maintainability:** automated CI/CD, infrastructure as code, standard logging / metrics / tracing, runbooks and architecture decision records (ADRs) keep the system understandable as the team changes.